

LEGALEASEAI

AI-Assisted Legal Document Drafting System

PROJECT DOCUMENTATION

Prepared & Developed by
DHARSHINI

Professional Student Project • Python • FastAPI • Gemini AI

“From project idea → code → debugging → working AI API → final document”

01 | PROJECT OVERVIEW

Understanding what Dharshini built and why it exists

Project Vision

LegalEaseAI is a Python-based application designed to generate structured legal-document drafts from user-provided information. The system connects a frontend form to a FastAPI backend and then uses Google Gemini through the official Python SDK to generate the draft.

Core objective

Accept document details from the user in a simple interface.

Send the structured information to the FastAPI backend.

Build a controlled prompt from the submitted fields.

Generate a draft through Gemini AI.

Return the generated document text to the frontend.

Keep the workflow understandable, testable and maintainable.

Dharshini's role in the implementation

The project workflow documented here treats Dharshini as the developer responsible for assembling the application, configuring the environment, connecting the API layers, testing requests and fixing

runtime errors.

02 | PROJECT STRUCTURE

Know exactly where each part belongs

```
LegalEaseAI/  
  
  backend/  
    main.py  
    routes.py  
    __init__.py  
    ai_core/  
      __init__.py  
      gemini_generator.py  
  
  frontend/  
    app.py  
  
  .env  
  requirements.txt  
  # Developer: Dharshini
```

What each file does

backend/main.py – starts and exposes the FastAPI application.

backend/routes.py – contains the API route logic.

backend/ai_core/gemini_generator.py – connects the backend to Gemini and creates the AI prompt.

frontend/app.py – provides the user-facing interface.

.env – stores local configuration and the Gemini API key.

requirements.txt – records Python dependencies.

Simple mental model

Frontend → FastAPI route → Gemini generator → Gemini API → generated draft → FastAPI response → Frontend.

03 | ENVIRONMENT & API KEY

Dharshini's local configuration layer

.env file

The real API key should never be placed in documentation, screenshots, GitHub repositories or chat messages. Because the key was previously visible during troubleshooting, it should be revoked/rotated and replaced with a new key before final submission.

.gitignore

```
.env
.venv/
__pycache__/
*.pyc
# Developer: Dharshini
```

Important

Use a placeholder in the report. Never paste the real secret into the report.

04 | GEMINI GENERATOR

The AI integration written by Dharshini

```
from google import genai
import os
from dotenv import load_dotenv
load_dotenv()
class GeminiDocumentGenerator:
    def __init__(self):
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError(
                "GEMINI_API_KEY not found in .env file"
            )
        self.client = genai.Client(api_key=api_key)
# Developer: Dharshini
```

What happens here?

load_dotenv() loads local environment variables.

GEMINI_API_KEY is read without hard-coding the secret.

genai.Client creates the Gemini client.

The class keeps AI generation logic separate from the API route.

05 | PROMPT CONSTRUCTION

Turning form data into an AI instruction

```

document_type = data.get("document_type", "Document")
jurisdiction = data.get("jurisdiction", "")
parties = data.get("parties", [])
terms = data.get("terms", "")
effective_date = data.get("effective_date", "")
language = data.get("language", "English")
prompt = f"""
Create a professional draft {document_type}.
Jurisdiction:
{jurisdiction}
Parties:
{parties}
Terms:
{terms}
Effective Date:
{effective_date}
Language:
{language}
Use only the information provided.
Do not invent missing names, addresses,
dates, or amounts.
Create a clear and properly structured document.
Mention that this is a draft requiring
legal review.
Developer: Dharshini
"""

```

Why these instructions matter

The prompt explicitly tells the model to use the supplied information and avoid inventing missing facts. It also makes the legal-review status clear, so the output is positioned as a draft rather than a substitute for professional legal advice.

06 | MODEL REQUEST & RESPONSE

The actual AI generation step

```

model = os.getenv(
    "GEMINI_MODEL",
    "gemini-3.8-flash"
)
response = self.client.models.generate_content(
    model=model,
    contents=prompt
)
return response.text
# Developer: Dharshini

```

Execution flow

Read the configured model name from .env.

Fall back to the configured current model name if the environment variable is absent.

Send the constructed prompt to Gemini.

Read response.text.

Return the generated draft to the API layer.

Successful runtime evidence

During debugging, the backend eventually returned HTTP 200 OK for POST /generate, confirming that the local API request completed successfully.

07 | TEMPORARY 503 FAILURE

Dharshini's debugging journey

```
for attempt in range(3):
    try:
        response = self.client.models.generate_content(
            model=model,
            contents=prompt
        )
        return response.text
    except Exception as e:
        error_message = str(e)
        if (
            "503" in error_message
            or "UNAVAILABLE" in error_message
            or "high demand" in error_message.lower()
        ):
            if attempt < 2:
                wait_time = 2 ** attempt
                time.sleep(wait_time)
                continue
            raise e
# Developer: Dharshini
```

Observed error

The API returned HTTP 500 because the upstream Gemini service returned HTTP 503 UNAVAILABLE with a high-demand message. This was different from the earlier local connection problem.

Debugging lesson

A 500 from the local backend does not automatically mean the Python code itself is broken. The underlying exception must be read to identify whether the failure is local, configuration-related, or caused by a temporary upstream service condition.

08 | INSTALLATION & VIRTUAL ENVIRONMENT

Preparing Dharshini's Python environment

```
PS C:\LegalEaseAI> .\venv\Scripts\python.exe -m pip install google-genai
```

The dependency check showed google-genai was installed in the project virtual environment. Using the virtual-environment Python executable avoids accidentally installing packages into a different Python installation.

```
PS C:\LegalEaseAI> .\venv\Scripts\python.exe -m uvicorn backend.main:app --reload --port 8000
```

Expected result

```
INFO: Uvicorn running on
http://127.0.0.1:8000
INFO: Application startup complete.
Developer: Dharshini
```

09 | STARTING THE BACKEND

The command Dharshini should run

Recommended working directory

Open the terminal at the LegalEaseAI project root — the folder containing backend, frontend, .env and requirements.txt.

```
PS C:\LegalEaseAI> .\venv\Scripts\python.exe -m uvicorn backend.main:app --reload --port 8000
```

Why this command is reliable

- It explicitly uses the project virtual environment.
- It starts backend.main:app from the project root.
- reload automatically restarts the server after code changes.
- port 8000 keeps the local API endpoint predictable.

Browser checks

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/openapi.json
# Developer test: Dharshini
```

10 | FRONTEND → API REQUEST

How the complete request travels

User fills the LegalEaseAI form

frontend/app.py

POST http://127.0.0.1:8000/generate

backend/routes.py

GeminiDocumentGenerator

Gemini API

Generated draft

JSON response

Frontend

Developer flow: Dharshini

What Dharshini should verify

Backend terminal shows the POST request.

The response status is 200 for a successful generation.

The frontend displays the returned document.

A 500 response should be followed by checking the backend traceback.

11 | FASTAPI / DOCS

Testing the API without guessing

Swagger / OpenAPI

FastAPI automatically exposes interactive API documentation at /docs. This is useful for Dharshini because the endpoint can be inspected and tested independently from the frontend.

```
GET /docs
GET /openapi.json
POST /generate
# Developer: Dharshini
```

Known successful checks

GET /docs → 200 OK

GET /openapi.json → 200 OK

POST /generate → 200 OK after the temporary Gemini service issue cleared.

Why this is useful

If /docs works but the frontend cannot generate a document, the debugging focus can move to the frontend request or payload rather than assuming the entire backend is down.

12 | DEBUGGING HISTORY

From first error to working request

Stage	Observed problem	Resolution / lesson
1	WinError 10061 / connection refused	Backend was not listening on port 8000.
2	PowerShell execution policy	Used the project Python executable / adjusted environment workflow.
3	No module named backend	Started Uvicorn from the correct project root.
4	Gemini model 404	Updated the configured model from the unavailable older model.
5	Gemini 503 high demand	Retried after the upstream service temporarily reported high demand.
6	Final POST /generate 200 OK	Backend + Gemini integration completed successfully.

Key debugging principle

Read the deepest useful error message first. The same HTTP 500 can represent very different root

causes.

13 | GITHUB & PROJECT HYGIENE

How Dharshini should prepare the repository

Files to commit

```
backend/  
frontend/  
requirements.txt  
README.md  
.gitignore  
# Safe repository contents — Dharshini
```

Files that should stay local

```
.env  
.venv/  
__pycache__/  
*.pyc  
# Never commit secrets — Dharshini
```

Suggested commit sequence

```
git init  
git add .  
git commit -m "Initial LegalEaseAI project"  
git status  
# Developer: Dharshini
```

Before pushing, verify that .env is ignored and that no API key appears in source files, screenshots, documentation or Git history.

14 | END-TO-END TEST PLAN

Dharshini's final verification checklist

Check	Expected result
Virtual environment	Project Python executable runs correctly
Dependencies	google-genai and required packages are installed
Environment	.env contains a valid rotated key and model name

Backend

API docs

Generate

Frontend

Security

Uvicorn starts on 127.0.0.1:8000

/docs and /openapi.json return successfully

POST /generate returns 200 for a valid request

Generated text is displayed

No API key appears in GitHub or report

Dharshini's final run

```
PS C:\LegalEaseAI> .\venv\Scripts\python.exe -m uvicorn backend.main:app --reload --port 8000
```

Then open the frontend, submit a test document, and confirm that the backend terminal records POST / generate with a successful response.

15 | FINAL PROJECT SUMMARY

Professional handover for Dharshini

What the project demonstrates

Python project organization with separated frontend, backend and AI logic.

FastAPI-based API development.

Environment-variable based secret management.

Gemini API integration through google-genai.

Prompt construction from structured user input.

Real-world debugging across local and upstream service failures.

Basic retry handling for temporary service unavailability.

API testing through FastAPI Swagger/OpenAPI.

Repository hygiene and secret-protection practices.

Future development areas

Add stronger input validation and structured request models.

Add automated tests for routes and document generation.

Add persistent document history if required.

Improve error messages shown to end users.

Add authentication and role-based access if deployed publicly.

Add document export formats such as PDF/DOCX.

Review generated legal drafts with qualified legal professionals before real-world use.

Developer statement

This documentation records the implementation and debugging workflow of LegalEaseAI in a way that lets Dharshini explain not only what the project does, but also where each piece of code belongs, how it is executed, and how the major runtime errors were diagnosed.

THANK YOU

LegalEaseAI

Prepared & Developed by DHARSHINI

Project Documentation • Code • Execution • Debugging • Testing